

```
"""
```

K-Nearest Neighbors (KNN) Classification

```
=====
```

This script demonstrates the implementation of KNN classification using the Iris dataset. It covers data normalization, model training, parameter tuning, evaluation, and visualization of decision boundaries.

```
"""
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from matplotlib.colors import ListedColormap

# Load and prepare the dataset
def load_data():
    """Load Iris dataset and return features, target and feature names"""
    iris = load_iris()
    X = iris.data
    y = iris.target
    feature_names = iris.feature_names
    target_names = iris.target_names

    # Convert to DataFrame for easier manipulation
    df = pd.DataFrame(X, columns=feature_names)
    df['species'] = [target_names[i] for i in y]
```

```

print(f"Dataset shape: {df.shape}")
print(f"Feature names: {feature_names}")
print(f"Target classes: {target_names}")
print("\nSample data:")
print(df.head())

return X, y, feature_names, target_names

```

Normalize features

```
def normalize_features(X):
```

```
    """Normalize features using StandardScaler"""
```

```
    scaler = StandardScaler()
```

```
    X_normalized = scaler.fit_transform(X)
```

```
    print("\nBefore normalization:")
```

```
    print(f"Mean: {np.mean(X, axis=0)}")
```

```
    print(f"Standard deviation: {np.std(X, axis=0)}")
```

```
    print("\nAfter normalization:")
```

```
    print(f"Mean: {np.mean(X_normalized, axis=0)}")
```

```
    print(f"Standard deviation: {np.std(X_normalized, axis=0)}")
```

```
    return X_normalized, scaler
```

Train and evaluate model with different K values

```
def train_evaluate_models(X, y):
```

```
    """Train KNN models with different K values and evaluate them"""
```

```
    # Split the data
```

```
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.3, random_state=42
```

```
)
```

```

# Try different values of K
k_values = list(range(1, 31, 2)) # Odd values from 1 to 30
train_scores = []
test_scores = []
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)

    # Train the model
    knn.fit(X_train, y_train)

    # Evaluate on training data
    train_score = knn.score(X_train, y_train)
    train_scores.append(train_score)

    # Evaluate on test data
    test_score = knn.score(X_test, y_test)
    test_scores.append(test_score)

    # Cross-validation score
    cv_score = np.mean(cross_val_score(knn, X, y, cv=5))
    cv_scores.append(cv_score)

    print(f"K = {k}: Training accuracy = {train_score:.4f}, Test accuracy = {test_score:.4f}, CV accuracy = {cv_score:.4f}")

# Find the best K value based on CV score
best_k = k_values[np.argmax(cv_scores)]
print(f"\nBest K value: {best_k} with CV accuracy: {max(cv_scores):.4f}")

```

```
# Plot accuracy vs K values

plt.figure(figsize=(10, 6))

plt.plot(k_values, train_scores, '-o', label='Training Accuracy')
plt.plot(k_values, test_scores, '-o', label='Test Accuracy')
plt.plot(k_values, cv_scores, '-o', label='CV Accuracy')

plt.xlabel('K Value')
plt.ylabel('Accuracy')

plt.title('KNN Performance vs K Value')

plt.legend()

plt.grid(True)

plt.savefig('knn_k_values_performance.png')

plt.close()
```

```
return best_k, X_train, X_test, y_train, y_test
```

```
# Evaluate the model in detail
```

```
def detailed_evaluation(X_train, X_test, y_train, y_test, best_k):
```

```
    """Evaluate the best KNN model in detail"""
```

```
    # Train the model with the best K
```

```
    best_knn = KNeighborsClassifier(n_neighbors=best_k)
```

```
    best_knn.fit(X_train, y_train)
```

```
    # Make predictions
```

```
    y_pred = best_knn.predict(X_test)
```

```
    # Calculate accuracy
```

```
    accuracy = accuracy_score(y_test, y_pred)
```

```
    print(f"\nBest model accuracy: {accuracy:.4f}")
```

```
    # Confusion matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
# Classification report
```

```
report = classification_report(y_test, y_pred)
```

```
print("\nClassification Report:")
```

```
print(report)
```

```
# Visualize confusion matrix
```

```
plt.figure(figsize=(8, 6))
```

```
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
```

```
plt.title('Confusion Matrix')
```

```
plt.colorbar()
```

```
classes = ['setosa', 'versicolor', 'virginica']
```

```
tick_marks = np.arange(len(classes))
```

```
plt.xticks(tick_marks, classes, rotation=45)
```

```
plt.yticks(tick_marks, classes)
```

```
# Add text annotations
```

```
thresh = cm.max() / 2.
```

```
for i in range(cm.shape[0]):
```

```
    for j in range(cm.shape[1]):
```

```
        plt.text(j, i, format(cm[i, j], 'd'),
```

```
                horizontalalignment="center",
```

```
                color="white" if cm[i, j] > thresh else "black")
```

```
plt.tight_layout()
```

```
plt.ylabel('True label')
```

```
plt.xlabel('Predicted label')
```

```
plt.savefig('knn_confusion_matrix.png')
```

```
plt.close()
```

```
return best_knn
```

```
# Visualize decision boundaries
```

```
def visualize_decision_boundaries(X, y, knn, feature_names, target_names):
```

```
    """Visualize the decision boundaries for 2D projections of the data"""
```

```
    # Create pairs of features
```

```
    feature_pairs = [(0, 1), (2, 3), (0, 2), (1, 3)]
```

```
    # Setup figure
```

```
    plt.figure(figsize=(15, 12))
```

```
    # Create a mesh to plot decision boundaries
```

```
    h = 0.02 # Step size in the mesh
```

```
    cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
```

```
    cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])
```

```
    # Plot decision boundaries for each pair of features
```

```
    for i, (f1, f2) in enumerate(feature_pairs):
```

```
        plt.subplot(2, 2, i+1)
```

```
        # Select only the two features we want to visualize
```

```
        X_subset = X[:, [f1, f2]]
```

```
        # Fit KNN on this subset
```

```
        knn_subset = KNeighborsClassifier(n_neighbors=knn.n_neighbors)
```

```
        knn_subset.fit(X_subset, y)
```

```
        # Create mesh grid
```

```
x_min, x_max = X_subset[:, 0].min() - 1, X_subset[:, 0].max() + 1
y_min, y_max = X_subset[:, 1].min() - 1, X_subset[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
```

```
# Predict for each point in the mesh
Z = knn_subset.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
# Plot the decision boundary
plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.8)
```

```
# Plot the training points
scatter = plt.scatter(X_subset[:, 0], X_subset[:, 1], c=y,
                     cmap=cmap_bold, edgecolor='k', s=50)
```

```
plt.title(f'Decision Boundary: {feature_names[f1]} vs {feature_names[f2]}')
plt.xlabel(feature_names[f1])
plt.ylabel(feature_names[f2])
plt.tight_layout()
```

```
plt.legend(handles=scatter.legend_elements()[0], labels=target_names)
plt.savefig('knn_decision_boundaries.png')
plt.close()
```

```
# Main function to execute the workflow
```

```
def main():
```

```
    # Load the dataset
```

```
    X, y, feature_names, target_names = load_data()
```

```
    # Normalize features
```

```
X_normalized, scaler = normalize_features(X)

# Train and evaluate models with different K values
best_k, X_train, X_test, y_train, y_test = train_evaluate_models(X_normalized, y)

# Detailed evaluation of the best model
best_knn = detailed_evaluation(X_train, X_test, y_train, y_test, best_k)

# Visualize decision boundaries
visualize_decision_boundaries(X_normalized, y, best_knn, feature_names, target_names)

print("\nKNN Classification task completed successfully!")
print("Generated visualizations have been saved to disk.")

if __name__ == "__main__":
    main()
```